Jewellery Manufacturing SaaS — V1 Spec

Nov 8, 2026 · @Yash

Overview

V1 is a multi-tenant SaaS that lets a gold jewellery manufacturer account for every gram of gold across kaarigars, run orders through production, and take B2B orders from retailers online.

Who uses it

Role	Where	What they do
Owner / admin	Manufacturer app	Sets up karats, processes, wastage limits, staff, retailers; sees reports
Store / metal manager	Manufacturer app	Issues and receives gold to kaarigars, records scrap and loss
Production manager	Manufacturer app	Turns orders into work orders, moves job cards through stages
Sales / accounts	Manufacturer app	Approves retailer orders, rates, dispatch
Kaarigar	Kaarigar app (PWA)	Sees own jobs and gold balance, confirms issues, submits returns with photos, sees labour earned
Retailer	B2B portal	Browses designs, places orders, tracks status, sees account

In V1: gold ledger and kaarigar tracking, orders and work orders, B2B retailer portal, full accounting with GST billing, kaarigar app, hallmarking (HUID) tracking, installable PWA, tenant onboarding and billing.

Not in V1: WhatsApp AI assistant, diamond/stone inventory beyond weight, POS for showrooms, app-store apps (the PWA covers mobile).

Architecture

One Laravel API serves two React front ends (manufacturer app and retailer portal) from a single database where every row carries a `tenant_id`.

Layer	Choice	Why
Backend	Laravel 12, REST API	Your existing stack; queues, policies, migrations built in
Front end	React + TypeScript + Vite, TanStack Query, shadcn/ui	Two apps in one monorepo sharing a UI package
Auth	Laravel Sanctum (SPA cookie auth), separate guards for staff, kaarigars (phone OTP) and retailers	Retailers must never reach manufacturer routes
Roles	spatie/laravel-permission, scoped per tenant	Owner, metal manager, production, sales, custom roles
Database	MySQL / MariaDB from cPanel	Weights as `DECIMAL(12,3)` grams, purity as `DECIMAL(6,3)` %, never floats
Multi-tenancy	Single DB, `tenant_id` + global scope (stancl/tenancy single-DB mode or your own trait)	Cheapest to run and migrate; move big tenants to their own DB later
Tenant routing	Subdomain: `acme.yourapp.in` (staff), `acme.yourapp.in/portal` (retailers)	Needs a wildcard subdomain and wildcard SSL in cPanel; otherwise add each tenant's subdomain by hand
Files	Hosting disk (storage/app, served via the public symlink); S3-compatible storage later	Images resized on upload with Intervention Image (GD); move to S3 when disk runs short
Jobs	Database queue driver, run by cPanel cron every minute	PDFs, notifications, daily balance snapshots
Hosting	Your cPanel hosting, PHP 8.3 (Laravel 12 needs 8.2+)	Enough to build and run the pilots; move to a VPS when tenants and load grow

Running on cPanel (PHP 8.3)

Shared cPanel hosting can't run Redis, long-running workers or Node on the server, so the stack is set up to need none of them.

- **Document root:** point the domain (and `*.yourapp.in`) at Laravel's `/public` folder in cPanel → Domains. If the host won't allow that, keep the app outside `public_html` and put only `/public`'s contents there with an edited `index.php`.

- **PHP extensions** (cPanel → Select PHP Version): bcmath, ctype, fileinfo, gd, intl, mbstring, openssl, pdo_mysql, tokenizer, xml, zip, and gmp for Web Push.
- **Cron** (cPanel → Cron Jobs), every minute: `php artisan schedule:run`. The scheduler also runs `queue:work --stop-when-empty --max-time=50`, which handles PDFs, notifications and imports without a resident worker.
- **Cache and sessions:** `database` or `file` drivers instead of Redis.
- **PDFs:** barryvdh/laravel-dompdf (pure PHP; no headless Chrome needed). **Excel:** openspout or maatwebsite/excel.
- **Front end:** React is built on your machine or in GitHub Actions with `npm run build`; only the built files in `public/build` go to the server. Node never runs on the hosting.
- **Deploy:** GitHub Actions builds the assets, runs `composer install --no-dev`, and uploads over SSH (or FTP if SSH is off), then runs `php artisan migrate --force` and the config/route/view cache commands. cPanel's Git Version Control works too if you build assets before committing.
- **SSL:** AutoSSL for the main domain. Wildcard SSL for tenant subdomains often needs DNS validation (e.g. Cloudflare in front), so check what your host supports. If it can't, launch with path-based tenants (`yourapp.in/acme`) and switch later.
- **Backups:** cPanel backups plus a nightly `mysqldump` from cron, kept off the server (e.g. Google Drive or S3). Gold and accounting data can't be lost.
- **Limits to watch:** PHP max_execution_time and memory on large reports, inode/disk quotas for design photos, and the host's rules on outbound mail volume. Heavy reports run as queued jobs and email or notify when ready.

Rules that protect the gold data

- Ledger rows are append-only. Mistakes are fixed with a reversing entry, never an edit or delete.
- Every write goes through a DB transaction and is logged with user, time and IP (spatie/laravel-activitylog).
- Balances are always derived from transactions; cached balance tables are rebuilt nightly and checked against the sum.
- Tests on every money/metal calculation before anything ships.

Module 1 — Gold ledger & kaarigar tracking

Every gram is tracked as **fine gold** (pure 24K equivalent), so metal of any karat can be added, compared and balanced across kaarigars, processes and orders.

Fine weight = gross weight × purity % ÷ 100

Purity is stored as touch % (e.g. 22K = 91.6, 18K = 75.0), and each tenant can define its own karat list and tunch values.

Locations that hold gold. The ledger is double-entry: every transaction moves fine weight from one location to another, so the system total never changes except by purchase, sale or recorded loss.

- Vault / main stock
- Each kaarigar (an account per person or karkhana)
- Each work order in progress
- Scrap & dust pool (to refine later)
- Finished goods stock, and lots out at the hallmarking centre (AHC)
- Retailer metal accounts (gold deposited by retailers against orders)
- Loss account (approved wastage)

Transaction types

Type	From → To	Recorded by	Notes
Purchase / opening	Outside → Vault	Admin	Bullion or old gold with tunch report
Melting / alloying	Vault → Vault	Metal manager	24K + alloy in, 22K bar out; gross changes, fine must match
Issue	Vault or WO → Kaarigar	Metal manager	Linked to a job card, process step and WO
Receive	Kaarigar → WO / Finished goods	Metal manager	Gross, stone weight, net weight, purity
Scrap / dust return	Kaarigar → Scrap pool	Metal manager	Filings, sweepings, broken pieces
Loss write-off	Kaarigar → Loss	Owner approval	Only within allowed wastage, else flagged
Retailer deposit	Outside → Retailer account	Sales	Fine gold credited against future orders
Dispatch	Finished goods → Retailer	Sales	Reduces retailer metal balance or creates a bill
Refining	Scrap pool → Vault	Admin	Refiner's return; difference posted as refining loss

Closing a job card. When a kaarigar returns work, the system calculates:

Loss = fine issued − fine returned in work − fine returned as scrap

- Each process (melting, wire/sheet, casting, making, stone setting, filing, polishing, hallmarking) has an **allowed wastage %** set by the owner, per karat if needed.
- Loss within the limit is posted to Loss automatically; loss above the limit stays on the kaarigar's balance and raises an alert.
- Example from the reel: Ramesh is issued 100 g, returns 98.4 g; Imran is issued 80 g, returns 79.1 g; Suresh still holds 60 g on the bench.

What owners see

- Live balance per kaarigar: issued, returned, scrap, loss, pending (in grams fine and gross)
- Wastage % per kaarigar and per process over any date range, so the worst performers stand out
- Day-book of all movements, and a daily closing that must balance to zero difference
- Printable issue/receive slips (PDF, with signature line) for each transaction

Edge cases to handle

- Stones and findings: gross weight includes stones; net gold = gross − stone weight − non-gold findings.
- Partial returns: one issue can close over several receives.
- Kaarigar advances: gold kept by a kaarigar across days carries forward as an open balance.
- Precision: store to 3 decimals (milligram); show 3 decimals in ledgers, 2 on the portal.

Module 2 — Orders & work orders

A retailer order becomes one or more work orders (WOs). Each WO moves through process stages as job cards, and every job card is tied to the gold issued for it.

Order lifecycle

1. **Placed** — from the portal, or entered by sales for phone/WhatsApp orders.
2. **Confirmed** — sales checks design, size, karat, quantity, target weight, delivery date and the retailer's metal/credit position.
3. **In production** — the production manager creates WOs (split by design or batch; e.g. WO-3083 = 50 Classic kade).
4. **Ready** — all WOs passed QC and hallmarking; finished pieces with actual weights are in finished goods.
5. **Dispatched** — a delivery challan/invoice is generated from actual weights.

6. **Closed** — retailer metal and cash settled.

Side states: on hold, partially dispatched, cancelled (with the reason).

Work order

- Header: WO number, linked order line(s), design, karat, quantity, sizes, target weight per piece, due date, priority.
- Route: the ordered list of process stages for that design (a template per product type that can be edited per WO).
- Job cards: one per stage per kaarigar, each holding its issue/receive transactions and the loss for that stage.
- Progress: pieces and weight at each stage, so the board shows exactly where each order is stuck.

Production board

- Kanban columns = stages; cards = WOs with due date, kaarigar and weight in hand.
- Overdue and over-wastage cards are highlighted.
- One-click “issue for next stage” pre-fills weight from the previous receive.

Order line fields: design, karat, size, quantity, target weight (min–max), stone details, remarks, reference image.

Order numbering is per tenant and per financial year (e.g. `ORD/26-27/0001`).

Module 3 — B2B retailer portal

Retailers get a branded, mobile-first portal on the manufacturer's subdomain where they browse designs, order in bulk and track every order without calling.

Retailer accounts

- Created by the manufacturer, or self-signup that needs approval (GSTIN, shop name, city, contact).
- Each retailer can have several users (owner, staff).
- Per-retailer settings: visible catalogue groups, making-charge slab, credit limit, metal account on/off.

Catalogue

- Designs with photos, design code, category (kada, chain, bangle, ring…), karats available, approximate weight range, sizes, making charge.
- Filters by category, weight range, karat; search by design code.
- Collections/tags (new arrivals, temple, bridal) set by the manufacturer.

- Optional: show today's gold rate set by the manufacturer, and an estimated value per piece.

Ordering

- Cart with design × karat × size × quantity, weight preference and remarks per line.
- Order summary with estimated fine gold and making charges; final billing uses actual weights.
- Re-order from a past order in one tap.

Tracking & account

- Order status timeline that mirrors production: received → in production → quality check → ready → dispatched.
- Account page: metal balance (fine gold deposited vs used), outstanding amount, invoices and challans as PDF.
- Notifications by email and SMS in V1; WhatsApp in the next phase.

Manufacturer side of the portal: a design manager (bulk upload with images and CSV), retailer approvals, an incoming-orders queue, and a rate setting.

Module 4 — Full accounting

The books are double-entry and keep two columns on every party and voucher: **metal (fine grams)** and **money (₹)**. Owners can see how much gold and how much cash each retailer, supplier and kaarigar owes in one ledger, so a separate Tally setup isn't needed.

Parties

Party	Metal balance	Money balance
Retailer (debtor)	Gold deposited vs gold billed	Invoices vs receipts
Bullion dealer / supplier (creditor)	Gold bought on metal terms	Purchase bills vs payments
Kaarigar	Gold in hand (from Module 1)	Labour earned vs advances and payments
Refiner	Scrap sent vs fine returned	Refining charges
AHC (hallmark centre)	—	Hallmarking charges

Chart of accounts comes pre-built for a jewellery manufacturer and can be edited: cash, bank, gold stock (vault, WIP, finished), debtors, creditors, GST input/output, retailer metal deposits, sales, making-charge income, job-work income, kaarigar labour, wastage/loss, refining, hallmarking, salaries, rent and other expenses.

Vouchers

Voucher	What it does	Posted from
Sales invoice	Net weight × rate + making + stone + hallmark charges + GST; reduces finished stock	Dispatch (Module 2)
Purchase	Bullion or old gold in, at rate or on metal terms	Manual
Receipt / payment	Cash, bank, UPI, cheque against a party	Manual
Metal receipt / payment	Gold received from or given to a party, no money	Manual or portal deposit
Rate-cut (fixing)	Converts a party's metal balance into money at an agreed rate on a date	Manual, owner approval
Kaarigar labour bill	Labour from closed job cards; optional deduction for wastage over the limit, valued at that day's rate	Auto from job cards
Metal issue / receive	Gold movements, valued at cost for stock	Auto from Module 1
Journal, contra, credit/debit note	Adjustments, returns, cash–bank transfers	Manual

Billing and GST. Making charges can be per gram, per piece or a % of metal value, set per retailer slab. GST rates and HSN codes (7113 for gold jewellery) live in settings, not in code. The CA confirms them per tenant, including job-work cases. E-invoices (IRN + QR) are generated through a GSP API for tenants whose turnover requires it.

Kaarigar labour. Each process has a labour rate per gram or per piece, which can differ by kaarigar. Closing a job card adds labour to his account automatically. Advances and weekly payments are recorded against it, and he sees the same figures in the kaarigar app.

Reports: day book, cash and bank book, party ledger (metal + money), outstanding and ageing, trial balance, P&L, balance sheet, stock valuation, GSTR-1 and GSTR-3B summaries (Excel/JSON for filing), and a Tally XML export for CAs who want it.

Controls: voucher numbers per financial year, period lock after the books are closed, year-end carry-forward of metal and money balances, and every auto-posted voucher links back to its source transaction.

Module 5 — Kaarigar app

Each kaarigar gets a simple phone app (part of the PWA) that shows only his own jobs, gold and money, and lets him confirm what he receives and report what he returns. The store still records the final weights.

Login and language: phone number + OTP, no password. Hindi and English first, with Bengali and Gujarati added as tenants need them. Big buttons, icons and weights in large digits.

What he sees and does

Screen	What it shows	His action
Home	Gold in hand (fine and gross), jobs due today, labour earned this week	—
New issue	Gold the store just issued: weight, karat, WO, design photo, process, due date	**Accept** or **Dispute** (with a reason); both sides then hold the same record
My jobs	Open job cards with due dates and design references	Mark “started” or “ready to return”
Return	Pre-filled job card	Enters work, scrap and pieces, with photos of the work and the scale reading; the store checks and posts the real receive
Gold khata	Every issue, return, scrap and loss, with a running balance	Download statement (PDF)
Earnings	Labour per job, advances, payments, balance due	Request an advance
Messages	Notices from the owner, e.g. design changes	Read

Rules

- A kaarigar's entry is a request, never a ledger posting. Only the metal manager posts receives, so nothing changes without the store weighing it.

- An issue the kaarigar hasn't accepted within a set time shows as "unconfirmed" on the owner's dashboard.
- An outside karkhana gets one master login and can add its own workers; the gold balance stays with the karkhana.
- Kaarigars without smartphones keep working on paper slips. The store still enters everything, and slips carry a QR code that opens his statement.

Module 6 — Hallmarking (BIS HUID)

The software can't hallmark anything itself. Hallmarking is done by a BIS-recognised Assaying & Hallmarking Centre (AHC). The system tracks each lot sent to the AHC, the gold while it is there, and the HUID each piece comes back with.

How hallmarking works today

1. **Register once with BIS.** Any manufacturer, wholesaler or retailer selling hallmarked gold must hold a BIS jeweller registration, applied for online through NSWS / Manak Online (BIS registration guide).
2. **Raise a request.** On manakonline.in → Hallmarking → New Request, the jeweller picks an AHC and enters item category, quantity, weight and declared purity, then submits to the AHC. An AHC can also start the request without the portal (BIS FAQ).
3. **AHC tests and marks.** Pieces that pass are laser-marked with three marks: the BIS logo, the purity grade (e.g. 22K916) and a 6-character alphanumeric HUID that is unique to that piece (GJEPC). BIS guidelines revised in June–July 2026 also make AHCs photograph each article and record its weight (Agile Regulatory).
4. **Sell.** Hallmarking is mandatory at the first point of sale in the manufacturer → wholesaler → retailer chain, so a registered manufacturer may hold unhallmarked stock but must hallmark before selling in a mandatory district (BIS). Buyers verify a piece in the BIS Care app with "Verify HUID".

Gold grades covered: 9K, 14K, 18K, 20K, 22K, 23K and 24K.

How the system handles it

- **Hallmark lot:** pick QC-passed finished pieces → choose the AHC and declared purity → the system totals pieces and weight per category and prints a lot sheet that matches what staff type into Manak Online.
- **Gold at the AHC:** sending the lot moves its fine weight to a new ledger location, "AHC", so the gold never disappears from the books while it is out.
- **Return:** for each piece, staff record pass/fail, the HUID and the returned weight. HUIDs can be typed, read from a photo of the mark, or imported from the AHC's Excel list. Weight lost to testing samples posts to a "hallmark loss" account.

- **Failed pieces** go back into a rework work order with the AHC's remark.
- **Checks:** a HUID must be 6 alphanumeric characters and unique per tenant; a piece can't be dispatched to a retailer in a mandatory district without one.
- **Where the HUID shows:** piece tags/labels, challans, sales invoices, the retailer portal's order detail, and the finished-goods register.
- **Charges:** the AHC's bill posts to the AHC party account in accounting.

V1 does not connect to the BIS portal directly. Staff raise the request on Manak Online and the system records everything around it. If BIS opens an integration route for jewellers, it can be added later.

PWA

All three front ends (staff app, kaarigar app, retailer portal) are installable PWAs, so users add them to the home screen with no app-store release. Each tenant gets its own name, logo and colours on the icon.

Part	How
Install	`vite-plugin-pwa` (Workbox); a manifest generated per tenant and per app (`/app` , `/kaarigar` , `/portal` scopes); custom "Install app" prompt on Android; Add-to-Home-Screen guide on iPhone
Offline reading	App shell cached; the last-synced kaarigar khata, jobs, catalogue and orders readable without network (IndexedDB via TanStack Query persistence)
Offline writes	Only low-risk actions queue offline: kaarigar accept/return requests, the retailer cart, photos. They sync with an idempotency key when back online and show "pending" until the server confirms
Gold postings	Always online. An issue, receive or voucher counts as done only after the server confirms it, so balances can never fork
Notifications	Web Push (VAPID) via `laravel-notification-channels/webpush` : new issue for a kaarigar, order status for a retailer, wastage alert for the owner. iPhone users get push only after installing the app

Part	How
Camera	Photos on returns and QC, QR scan on slips and tags, HUID photo capture
Weighing scale	Store PC on Chrome reads the electronic scale over USB/serial (Web Serial API), so weights aren't typed by hand; manual entry stays as fallback
Printing	Issue/receive slips and tags through the browser print dialog or a thermal printer; PDFs for statements and invoices
Updates	New versions load in the background; "Update available" prompt so nobody is stuck on an old build

If a store later needs things a PWA can't do well (a Bluetooth printer on iPhone, deep background sync), the same React code can be wrapped with Capacitor for the app stores.

Database schema

About 45 tables cover V1. Every table except `tenants` and `plans` carries `tenant_id`, `created_by` and timestamps; weights are grams `DECIMAL(12,3)` and purity is `DECIMAL(6,3)`.

Platform & access

Table	Key columns	Relations
tenants	name, subdomain, gstin, logo, plan_id, status, trial_ends_at	has many users, retailers
plans / subscriptions	plan limits (users, retailers, designs), price, billing cycle, status	tenant → subscription
users	name, phone, email, password, is_active	spatie roles per tenant
retailers	shop_name, gstin, city, credit_limit, making_slab_id, metal_account_enabled, status	has many retailer_users, orders
retailer_users	retailer_id, name, phone, email, password	separate auth guard

Masters

Table	Key columns	Relations
karats	label (22K), purity_pct (91.6)	used by designs, transactions
processes	name, sequence, default_wastage_pct	route templates, job cards
process_wastage_rules	process_id, karat_id, allowed_wastage_pct	overrides per karat
kaarigars	name, phone, type (in-house / outside), address, is_active	ledger location
categories	name, parent_id	designs
designs	code, name, category_id, approx_weight_min/max, sizes (json), making_charge, making_type (per g / %), is_published	has many design_images, karats
design_images	design_id, path, sort	—
route_templates / route_template_steps	category_id, process_id, sequence	copied onto each WO
gold_rates	date, karat_id, rate_per_g	portal estimates

Gold ledger

Table	Key columns	Relations
metal_locations	type (vault, kaarigar, wo, scrap, finished, ahc, retailer, loss), ref_type, ref_id	polymorphic to kaarigar / WO / retailer
metal_transactions	voucher_no, type, date, from_location_id, to_location_id, job_card_id, remarks, reversed_by_id	header, append-only
metal_transaction_lines	transaction_id, karat_id, gross_wt, stone_wt, net_wt, purity_pct, fine_wt, pieces	one or more lines per voucher
location_balances	location_id, karat_id, fine_wt, gross_wt, as_of	cache, rebuilt from lines

Orders & production

Table	Key columns	Relations
orders	order_no, retailer_id, source (portal / staff), status, due_date, remarks	has many order_lines
order_lines	order_id, design_id, karat_id, size, qty, target_wt_min/max, remarks, status	linked to WOs
work_orders	wo_no, design_id, karat_id, qty, due_date, priority, status	many-to-many order_lines via wo_order_lines
wo_steps	work_order_id, process_id, sequence, status	route copied from template
job_cards	wo_step_id, kaarigar_id, issued_fine, returned_fine, scrap_fine, loss_fine, status, closed_at	has metal_transactions
finished_items	work_order_id, design_id, karat_id, size, gross_wt, net_wt, huid, status	dispatched via dispatch_lines
dispatches / dispatch_lines	retailer_id, challan_no, date / finished_item_id	invoices later

Accounting

Table	Key columns	Relations
financial_years	start_date, end_date, locked_until	vouchers
account_groups / accounts	name, code, nature (asset, liability, income, expense), is_system	voucher_entries
parties	type (retailer, supplier, kaarigar, refiner, ahc), ref_type, ref_id, gstin, state_code	links to retailers / kaarigars / ahcs
vouchers	voucher_no, type, date, financial_year_id, party_id, narration, source_type, source_id, status	auto or manual
voucher_entries	voucher_id, account_id, party_id, debit, credit, fine_dr, fine_cr	money and metal on every line

Table	Key columns	Relations
invoice_lines	voucher_id, finished_item_id, huid, net_wt, rate, metal_value, making, stone_charge, hallmark_charge, hsn, gst_rate, cgst, sgst, igst	sales and purchase bills
tax_settings	hsn, description, gst_rate, effective_from	billing
labour_rates	process_id, kaarigar_id (nullable), basis (per g / per pc), rate	kaarigar labour bills
einvoices	voucher_id, irn, ack_no, qr, status	GSP response

Kaarigar app & PWA

Table	Key columns	Relations
kaarigar_users	kaarigar_id, phone, otp_hash, language, last_seen_at	kaarigar guard
kaarigar_requests	type (accept, dispute, return, advance), job_card_id, weights (json), status, reviewed_by	becomes a ledger posting only when the store approves
attachments	attachable_type, attachable_id, path, kind (work photo, scale photo, hallmark photo)	job cards, requests, lot items
push_subscriptions	user_type, user_id, endpoint, keys	Web Push

Hallmarking

Table	Key columns	Relations
ahcs	name, bis_recognition_no, city, contact	party for charges
hallmark_lots	lot_no, ahc_id, declared_purity, pieces, gross_wt, sent_at, returned_at, status, charges	metal location “ahc”
hallmark_lot_items	lot_id, finished_item_id, sent_wt, returned_wt, result (pass/fail), huid, remarks	updates finished_items.huid

Audit: `activity_log` (spatie) for every change, plus `notifications`.

Screens

V1 comes to about 45 screens across the manufacturer app, the kaarigar app and the retailer portal.

Area	Screen	Main users
Manufacturer	Dashboard: gold in vault / with kaarigars / in WIP, today's issues and receives, overdue WOs, wastage alerts	Owner
Manufacturer	Issue gold (job card picker, weight, karat, slip print)	Metal manager
Manufacturer	Receive gold (work + scrap + stone weight, auto loss calculation)	Metal manager
Manufacturer	Kaarigar list and kaarigar ledger (running balance, wastage %)	Owner, metal manager
Manufacturer	Vault stock, scrap pool, melting/alloying, refining	Metal manager
Manufacturer	Day-book and daily closing	Owner
Manufacturer	Orders list, order detail, new order (staff entry)	Sales
Manufacturer	Work orders list, WO detail with route and job cards	Production
Manufacturer	Production board (kanban by stage)	Production
Manufacturer	Finished goods, QC/hallmark entry, dispatch and challan	Production, sales
Manufacturer	Designs and catalogue manager (bulk upload)	Sales
Manufacturer	Retailers (approve, settings, metal account)	Sales
Manufacturer	Reports: wastage by kaarigar/process, order ageing, stock summary	Owner
Manufacturer	Settings: karats, processes, wastage rules, route templates, users and roles, gold rate, branding	Admin
Retailer portal	Login / signup, catalogue, design detail, cart, order confirmation	Retailer
Retailer portal	My orders, order tracking timeline, account (metal and money balance, PDFs)	Retailer

Area	Screen	Main users
Platform	Tenant signup and onboarding wizard, billing, super-admin panel	You

Screens for accounting, kaarigar app and hallmarking

Area	Screen	Main users
Accounting	Sales invoice, purchase, receipt/payment, metal receipt/payment, rate-cut, journal	Accounts
Accounting	Party ledger (metal + money), outstanding and ageing	Owner, accounts
Accounting	Kaarigar labour bills, advances and payments	Accounts
Accounting	Day book, cash/bank book, trial balance, P&L, balance sheet, GST reports, Tally export	Owner, CA
Accounting	Chart of accounts, tax settings, labour rates, financial year and period lock	Admin
Kaarigar app	Login (OTP), home, new issue (accept/dispute), my jobs, return with photos, gold khata, earnings, messages	Kaarigar
Manufacturer	Kaarigar requests inbox (approve returns, resolve disputes, advances)	Metal manager
Hallmarking	New hallmark lot, lots at AHC, lot return with HUID entry/import, HUID search	Production

API endpoints

Two route groups share one Laravel app: `/api/v1/*` for staff (staff guard, tenant resolved from the subdomain) and `/api/portal/v1/*` for retailers (retailer guard). Standard REST resources are not listed one by one.

Group	Endpoint	Purpose
Masters	`apiResource` karats, processes, wastage-rules, kaarigars, categories, designs, route-templates, retailers, users	CRUD

Group	Endpoint	Purpose
Ledger	`POST /metal/issue`	Issue gold to a kaarigar against a job card
Ledger	`POST /metal/receive`	Receive work + scrap; returns calculated loss and alert flag
Ledger	`POST /metal/transfer`, `/metal/melt`, `/metal/refine`, `/metal/deposit`	Vault moves, alloying, refining, retailer deposits
Ledger	`POST /metal/transactions/{id}/reverse`	Reversing entry (no edits or deletes)
Ledger	`GET /metal/balances?location_type=kaarigar`	Live balances
Ledger	`GET /kaarigars/{id}/ledger?from=&to=`	Kaarigar statement
Ledger	`GET /metal/daybook?date=`, `POST /metal/closing`	Day-book and daily closing
Orders	`apiResource` orders; `POST /orders/{id}/confirm`, `/hold`, `/cancel`	Order lifecycle
Production	`POST /orders/{id}/work-orders`	Create WOs from order lines
Production	`GET /work-orders/board`, `POST /work-orders/{id}/steps/{step}/advance`	Kanban board, move a stage
Production	`POST /job-cards`, `POST /job-cards/{id}/close`	Assign a kaarigar to a stage, close it
Production	`POST /finished-items`, `POST /dispatches`	Record finished pieces, dispatch + challan PDF
Reports	`GET /reports/wastage`, `/reports/order-ageing`, `/reports/stock`	Owner reports, CSV/PDF export
Portal	`POST /portal/v1/login`, `/register`	Retailer auth
Portal	`GET /portal/v1/designs`, `/designs/{code}`, `/gold-rate`	Catalogue
Portal	`GET/POST /portal/v1/cart`, `POST /portal/v1/orders`	Cart and ordering
Portal	`GET /portal/v1/orders`, `/orders/{id}`, `/account`	Tracking and account

Endpoints for the new modules (kaarigar routes under `/api/kaarigar/v1/*` with their own guard)

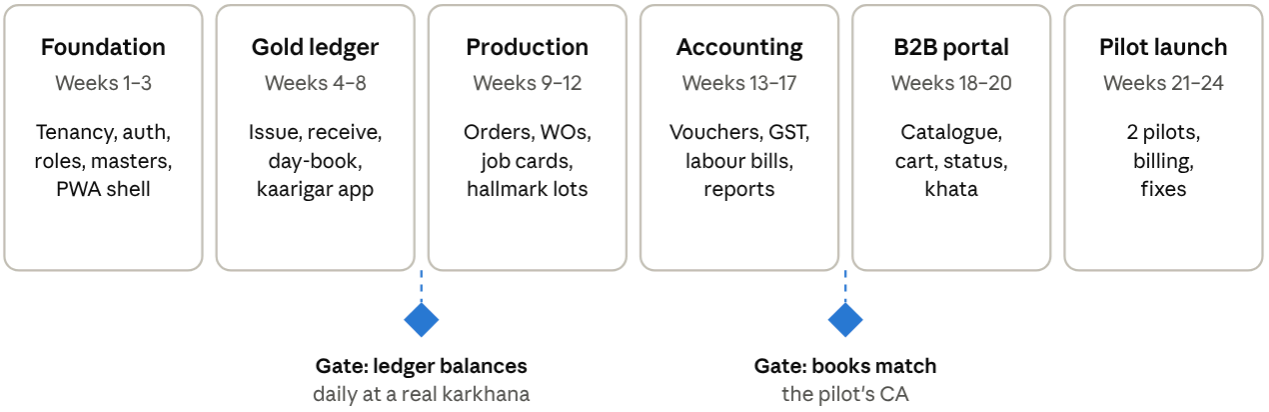
Group	Endpoint	Purpose
Accounting	`apiResource` accounts, parties, vouchers, tax-settings, labour-rates	CRUD; vouchers validate debit = credit and metal in = metal out
Accounting	`POST /vouchers/{id}/cancel`	Reversal voucher, respects period lock
Accounting	`POST /invoices/from-dispatch/{dispatch}`, `POST /invoices/{id}/einvoice`	Bill a dispatch, generate IRN
Accounting	`POST /rate-cuts`	Fix a party's metal balance into money
Accounting	`GET /reports/trial-balance`, `/pnl`, `/balance-sheet`, `/gstr1`, `/gstr3b`, `/parties/{id}/ledger`	Reports and exports
Kaarigar	`POST /kaarigar/v1/otp`, `/login`	Phone OTP auth
Kaarigar	`GET /kaarigar/v1/home`, `/jobs`, `/khata`, `/earnings`	His own data only
Kaarigar	`POST /kaarigar/v1/issues/{id}/accept`, `/dispute`; `POST /kaarigar/v1/jobs/{id}/return`; `POST /kaarigar/v1/advances`	Requests for the store to review
Kaarigar	`POST /kaarigar-requests/{id}/approve`, `/reject` (staff side)	Approval turns a return into a real receive
Hallmark	`apiResource` ahcs, hallmark-lots; `POST /hallmark-lots/{id}/send`, `/return`	Lot lifecycle and gold movement to and from the AHC
Hallmark	`POST /hallmark-lots/{id}/huids/import`	Import the AHC's HUID list (Excel/CSV)
PWA	`POST /push-subscriptions`, `GET /manifest/{app}.webmanifest`	Push and per-tenant manifests

Every write endpoint accepts an `Idempotency-Key` header, so a double-tap on a slow connection can't post the same gold issue twice.

Build roadmap

Build the gold ledger and kaarigar app first and prove them at one real manufacturer, then add production, accounting and the portal. The ledger is the reason owners will pay, and the hardest part to get right.

Six phases take V1 to paying pilots in about 24 weeks



V1 roadmap · 6 phases, 2 gates (estimate for one full-time developer)

Weeks are an estimate for one full-time developer; Claude can write much of the CRUD, tests and UI to shorten this.

SaaS plans. Price per tenant per month in tiers by staff users, retailers on the portal and designs in the catalogue, with a 14-day trial. Set the actual prices after talking to the pilot manufacturers.

After V1

- WhatsApp AI sales assistant: catalogue sharing, rate queries, order capture into the portal, follow-up reminders.
- App-store versions via Capacitor if a PWA limit shows up.
- Direct BIS portal integration for hallmark requests, if BIS opens one for jewellers.
- Bank statement import and auto-reconciliation.
- Barcode/RFID tags on finished pieces.
- Separate database for large tenants.